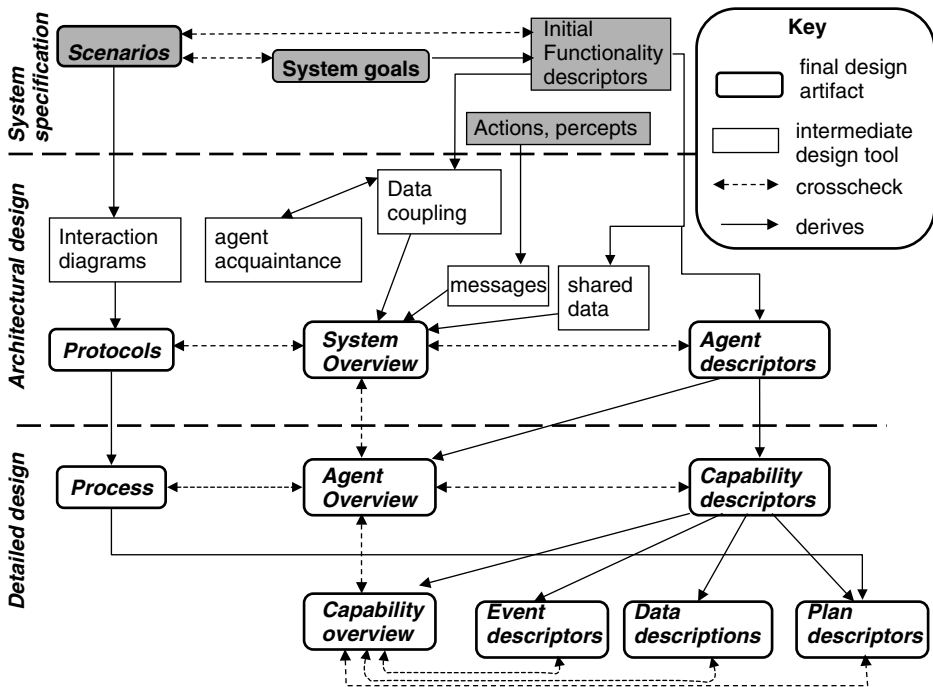


System Specification



Phases, artifacts and relationships in the design process

This chapter discusses in detail the artifacts and processes in the System Specification phase, the initial phase of the Prometheus methodology. Often, the initial documents from which a system analyst or system developer begins are simply a few paragraphs of loose description, or a rough understanding based on discussions and meetings. From this, a

clear and precise understanding of the system to be built must be developed. Preferably, it is developed in a manner that facilitates clear and traceable paths between the system specification phase and subsequent system development phases, such as design, implementation and testing.

Considerable work has been done in recent years on requirements engineering, recognizing that care and rigour at this early stage can avoid many more expensive problems at later stages. The Prometheus methodology focuses particularly on specification of goals, using some of the work of van Lamsweerde (e.g. (van Lamsweerde 2001)), and on scenario descriptions. In addition, it requires specification of *functionalities* (small chunks of behaviour) related to the identified goals. There is also a focus on the important issue of how the agent system interfaces with the environment in which it is situated, in terms of *percepts* that arrive, or can be obtained, from the environment, and *actions* that impact on the environment. As part of the interface specification, we also address interaction with any external data stores or information repositories.

The aspects that we develop in the system specification phase are then as follows:

1. Specification of system goals, resulting in a list of goals and sub-goals, with associated descriptors.
2. Development of a set of scenarios that have adequate coverage of the goals, and which provide a process-oriented view of the system to be developed.
3. Definition of a set of functionalities that are linked to one or more goals, and capture a limited piece of system behaviour, which can be described in a few sentences.
4. Description of the interface between the agent system and the environment in which it is situated, in terms of incoming percepts, outgoing actions and external information stores with which the system will interact.

These activities interact to support and complement each other, and the developer typically moves between them in a flexible manner. The productive interaction between scenario descriptions and goal elicitation has been explored increasingly in the requirements engineering literature in recent years (Liu and Yu 2001; Rolland *et al.* 1999). Our experience in agent system design is also that this interaction, together with specification of system interfaces, is natural and productive.

4.1 GOAL SPECIFICATION

The reasons for building the system should always be central in one's thinking when specifying the system, and thus *goals* are a natural construct to use in system specification. In addition, goals are central to the functioning of the intelligent software agents that are going to realize the system. The use of goals at the requirements engineering or system specification phase thus facilitates a mapping into later detailed design and implementation, as well as providing an appropriate mechanism for requirements specification.

4.1.1 IDENTIFY INITIAL GOALS

The initial brief system description usually contains some implicit indications of the system goals – what it is that the system is supposed to do. These provide a starting point for building an initial list of system goals. A simple underlining or highlighting of the relevant words or phrases in the description can be used to provide a first cut at the system goals.

For example, consider the following description:

We wish to develop a student enrolment system that allows students to enrol in subjects, add and delete subjects in accordance with rules and view their enrolment. Enrolment rules should be editable by authorized staff.

On the basis of this initial description, it is possible to extract the following system goals:

- Enrol student
- Add subject to enrolment
- Delete subject from enrolment
- Check enrolment against rules
- View enrolment
- Authenticate staff
- Edit rules.

These can then provide a basis for refinement, which provides an extended list of goals that can then be grouped into what we call *functionalities* – small chunks of system behaviour, describable in a few sentences.

In the rest of this chapter and the following chapters, we will develop an example of an electronic bookstore to illustrate the design process. To enable easy following of the example, we will enclose all of the example details in a framed box (which may extend over page breaks, in which case the bottom and top of the frame on the adjacent pages will be missing), as below. In addition, the collected details of the example can be found in Appendix A.

Electronic Bookstore: Case study

Here, we show an initial brief system description of an electronic bookstore and the set of system goals that can be extracted from that description:

We would like to develop a fully online system for worldwide sale of books. This system will offer a broad range of books to customers, and a personalized, friendly user interface. The system must facilitate fast and reliable service at all stages, from locating a desired book, to delivery of the purchase. The store should have competitive prices.

We identify our initial set of goals by highlighting parts of this description:

We would like to develop a **fully online system** for **worldwide sale of books**. This system will offer a **broad range of books** to customers, and a **personalized, friendly user interface**. The system must facilitate **fast and reliable service** at all stages, from **locating** a desired book, to **delivery** of the **purchase**. The store should have **competitive prices**.

This then yields the following extracted system goals:

- Worldwide sale of books
- Fully online system
- Broad range of books
- Personalized, friendly user interface
- Fast and reliable service
- Locating of books
- Delivery of books
- Purchase of books
- Competitive prices.

4.1.2 GOAL REFINEMENT

We now refine these goals somewhat using the simple technique of asking ‘how?’ as suggested by (van Lamsweerde 2001). We consider each goal and ask ‘how might this goal be achieved?’; the answers give the *sub-goals* of the goal under consideration. For example, the goal of providing worldwide sale of books could be achieved by having an online system and by delivering books internationally.

It would be possible to also use the abstraction techniques of asking ‘why?’, espoused by the same author, to build up a much more complete goal tree. However, we have chosen to limit the set of goals to those arising most directly.

As we refine the original goals to be slightly more specific, it is often the case that we find similar sub-goals arising under different initial goals. A grouping of similar sub-goals then provides the basis for what we call ‘functionalities’ – descriptions of limited chunks of system behaviour.

For example, refining the original goal of *fully online system*, we obtain sub-goals *find books online*, *pay online* and *order online*, while the original goal of *purchase books* leads to *find books*, *place order*, *make payment* and *arrange delivery*. *Pay online* and *make payment* are clearly closely related, if not identical, goals, and are therefore grouped together. Below we show the expanded goal set derived from our initial set of goals, and the initial arrangement of these into groupings.

Electronic Bookstore: Case study

Here, we have the expanded list of goals and associated sub-goals resulting from asking 'how?' and refining the initial list.

- Worldwide sale of books.
 - online system
 - deliver internationally
- Fully online system.
 - find books online
 - order online
 - pay online
- Broad range of books.
 - books from many publishers
 - update catalogue regularly
 - multiple suppliers
- Personalized, friendly user interface.
 - personalized welcoming
 - recommendations based on user profile
 - information available about orders in process
- Fast and reliable service.
 - arrange courier deliveries
 - provide estimates of delivery time
 - track delivery problems
 - have books available in stock
- Locating of books.
 - provide search facility
 - provide recommendations
- Delivery of books.
 - arrange delivery
 - monitor delivery
- Purchase books
 - find books
 - place order
 - make payment
 - arrange delivery
- Competitive prices.
 - set prices competitively
 - temporarily reduce prices to match competitors

After refining the goals, we can then start to rearrange them, moving similar goals together. As we do this we may find the same sub-goal under different parent goals, in which case we can coalesce them. We can then give a name to the grouping which describes it. This may be related to one of the original top level goals, or may be a new name.

Electronic Bookstore: Case study

Below is an initial re-arrangement of the goals with moved sub-goals shown in *italics*. The name chosen for the resulting grouping is shown in CAPITALS below each group of sub-goals. In some cases such as *competitive prices*, the sub-goals are divided into two functionalities.

- Worldwide sale of books
- Fully online system.
 - *online system* (from Worldwide sale of books)
 ONLINE INTERACTION
- Broad range of books.
 - books from many publishers
 - update catalogue regularly
 CATALOGUE MANAGEMENT
- Personalized, friendly user interface.
 - personalized welcoming
 WELCOMING
 - recommendations based on user profile
 - *provide recommendations* (from Locating of books)
 PROFILE MONITOR
- Fast and reliable service.
 - have books available in stock
 - *multiple suppliers* (from Broad range of books)
 STOCK MANAGEMENT
- Locating of books.
 - *find books online* (from Fully online system and also *find books* from Purchase books)
 - provide search facility
 BOOK FINDING
- Delivery of books.
 - *deliver internationally* (from Worldwide sale of books)
 - *arrange courier deliveries* (from Fast and reliable service)
 - arrange delivery (also includes *arrange delivery* from Purchase books)
 - monitor delivery
 - *provide estimates of delivery time* (from Fast and reliable service)

- *track delivery problems* (from Fast and reliable service)
- *information available about orders in process* (from Personalized, friendly user interface)

DELIVERY HANDLING

- Purchase books
 - place order
 - *order online* (from Fully online system)
 - make payment
 - *pay online* (from Fully online system)

PURCHASING

- Competitive prices.
 - set prices competitively

PRICE SETTING

- temporarily reduce prices to match competitors

COMPETITION MANAGEMENT

We continue to work with the list of goals and sub-goals, coalescing similar goals and adding goals as we see that a particular grouping is lacking some aspect. For example, the sub-goals of *deliver internationally*, *arrange courier delivery* and *arrange delivery* are coalesced into the single sub-goal *arrange delivery*.

As we do this, our original set of goals and sub-goals becomes a network of connected goals as shown in Figure 4.1. Each goal is represented by an oval and arrows join goals to sub-goals.

We also elaborate goals by adding more detailed sub-goals; for example, we add to PROFILE MONITOR the sub-goals of registering and updating the customer profile. In some cases, a sub-goal is replaced with a more detailed sub-goal (or collection of sub-goals). For example, under STOCK MANAGEMENT, we replace *have books available in stock* with *reorder stock*, and sub-goals to log outgoing and arriving books.

We aim for groupings of approximately two to five goals that belong together in a way that we can provide a brief but comprehensive description of the functionality to which these goals belong.

Some of our original goals may disappear if all sub-goals belonging to them are moved into other groupings. In our example, this happens for the original goal of *worldwide sale of books* as the sub-goals indicating how this goal was to be realized – *online system* and *worldwide delivery* – are moved. Similarly, some original goals may expand into more than one grouping, as is seen with *Delivery of books*, which leads to two groupings – one dealing with arranging deliveries and the other dealing with delivery problems. We may choose to retain goals such as *worldwide sale of books*, but mark them as *abstract*, indicating that they are a motivator for more concrete goals, but will not appear directly as the details are developed.

Below is the revised set of goal groupings for our example, organized by headings.

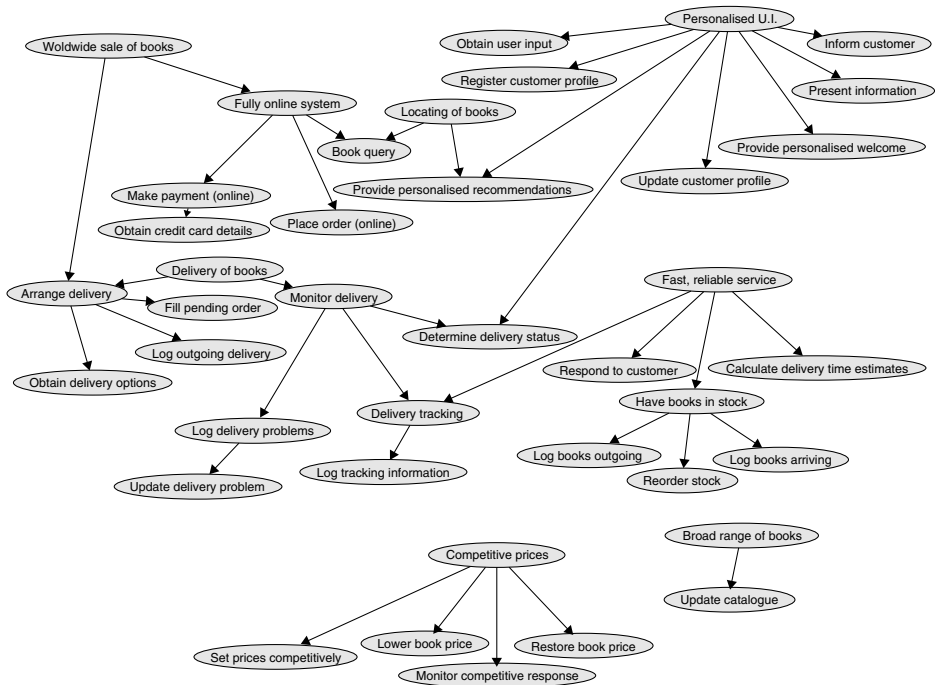


Figure 4.1 Goals for the electronic bookstore

Electronic Bookstore: Case study

The set of goal groupings that we developed after our initial pass at refining, coalescing and adding goals as needed:

- **ONLINE INTERACTIONS**
 - obtain user input
 - present information
- **CATALOGUE MANAGEMENT**
 - update catalogue

(note that a decision has been made at this point that the goal of having books from many publishers will be met outside the software system. The system will simply manage the catalogues it is given. Ensuring that these come from a wide variety of sources becomes the responsibility of the company using the system.)
- **WELCOMING**
 - provide personalized welcome
- **PROFILE MONITOR**
 - provide personalized recommendations

- register customer profile
- update customer profile
- STOCK MANAGEMENT
 - log books arriving
 - log books outgoing
 - reorder stock
- BOOK FINDING
 - book query
- DELIVERY HANDLING
 - obtain delivery options
 - calculate delivery time estimates
 - arrange delivery
 - determine delivery status
 - log outgoing delivery
- LOST GOODS MANAGEMENT
 - log delivery problem
 - request delivery tracking
 - respond to customer¹
 - log tracking information
- PURCHASING
 - purchase of books
 - place order
 - make payment
- PRICE SETTING
 - set prices
- COMPETITION MANAGEMENT
 - lower book price
 - monitor competitive response
 - restore book price

4.2 FUNCTIONALITIES

Functionality is the term we use for a chunk of behaviour, which includes a grouping of related goals, as well as percepts, actions and data relevant to the behaviour. A functionality should be coherent, in that it can be described adequately in one or two sentences, and can be named in a way that captures its essence. Functionalities allow for a mixture of both top-down and bottom-up design. They are identified by a top-down

¹... with the status of the goods ordered.

process of goal development. At the same time, they provide a bottom-up mechanism for determining the agent types and their responsibilities.

The process of refining and then grouping goals suggests an initial set of functionalities. Further work with scenarios and development of the specification may well suggest additional ones. System specification is an iterative process, starting with identification of goals, but then moving between scenarios, goals and functionalities, as well as identifying actions, percepts and data.

The groupings, namings and additions and merging of goals are continually refined. Figure 4.2 shows the current revised functionalities developed for the Electronic Bookstore at this stage. Functionalities are depicted by rectangles, goals with ovals, and actions with an action icon (a rectangle extended with a triangle pointing to the right). Arrows link functionalities to their goals and actions.

Once we have identified functionalities we can begin to develop *functionality descriptors*. Each functionality should be able to be described in a few sentences. If the description is larger than this, then the functionality should be split into multiple functionalities. In addition to a natural language description and information about the goals and actions that are included, the functionality descriptor should also include what we

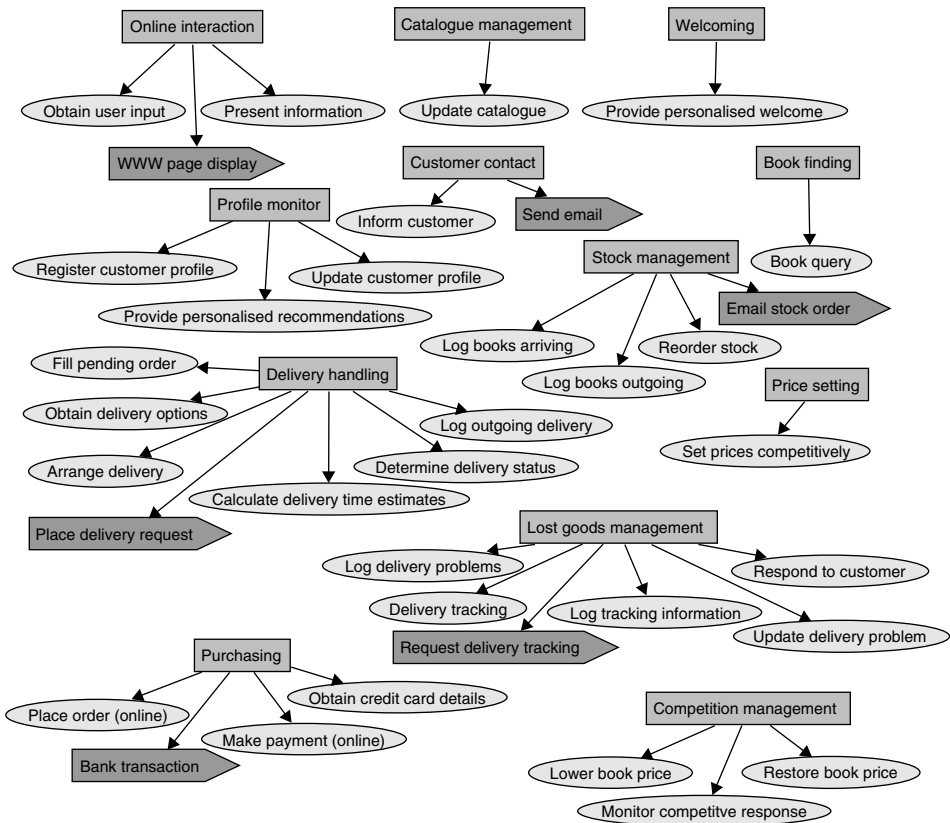


Figure 4.2 Functionalities for the electronic bookstore

call *triggers*: information about what events or situations will cause activity to be initiated within this functionality. These triggers may include identified percepts, but are not limited to these. Notes about the data required and produced by the functionality should also be included.

An example of a functionality descriptor for *Purchasing* is given below.

Electronic Bookstore: Case study

This is an example functionality descriptor that was developed during system specification, on the basis of the three goals grouped under *Stock Management*.

Stock Management Functionality

Description: This functionality monitors the stock available as it comes in and goes out, ordering new stock as needed. It maintains information as to when stock is expected to arrive.

Goals: Reorder stock, Log books arriving, Log books outgoing

Actions: E-mail stock order

Triggers: Stock arrival, Stock order delay, Failed stock arrival

Information used: Stock database, Customer order, Stock order

Information produced: Stock database, Delayed orders, Arrived orders

4.3 SCENARIO DEVELOPMENT

Scenarios are complementary to goals in that they show the sequences of steps that take place within the system. In developing goals, we are typically already building up scenarios of how these goals will be part of various processes within the system. Scenarios enable us to specify some of this structure, which in turn may help identify missing goals.

Scenarios are used primarily to illustrate the normal running of the system, although it can also be useful to develop some scenarios that indicate what is expected to happen when something goes wrong. As scenarios are developed, it becomes evident where there is a need for information from the environment (i.e. percepts) and where actions are required. Also, as scenarios are developed, it is common to identify additional goals that are needed. For example, in developing the **Query Late Books Scenario** in which the user is querying because their books have not arrived, we identified the need for a new goal *Update delivery problem*. We already had *Log delivery problem*, but discovered the need to update the information once the information from the tracking request had been received. Also in this scenario, we can see that two percepts and an action are required. The two percepts are the original user query from the web interface and the response to the tracking initiated, which are both input to the system in some way. The

action is the request for tracking of the delivery, which goes outside the system and initiates behaviour in the environment. Below we show the outline of this scenario made up of existing goals, the newly identified goal, the percepts and the action.

The core of a scenario consists of a sequence of steps. Possible steps are achieving a goal (GOAL), performing an action (ACTION), receiving a percept (PERCEPT), or referring to another use case scenario (SCENARIO). Additionally, we use the step type OTHER to cover unusual steps such as waiting for something to happen.

Electronic Bookstore: Case study

Here is the outline of the **Query Late Books Scenario** developed in the first round of scenario descriptions:

Query Late Books Scenario

Trigger: user enquiry

Description: When the user enquires about the status of their delivery the system requests that tracking be performed and waits for the result. In this scenario the result is that the book could not be located and so a replacement book is sent.

1. GOAL: Determine delivery status
2. GOAL: Log delivery problems
3. ACTION: Request delivery tracking
4. GOAL: Inform customer
5. OTHER: wait for response ...
6. PERCEPT: tracking info received
7. GOAL: Arrange delivery
8. GOAL: Log books outgoing
9. GOAL: Inform customer
10. GOAL: Update delivery problem

In the process of developing scenarios, it is common to find the need for new GOAL steps. These are then added to the appropriate functionality, if there is a natural place where they belong. It may be the case that there is a need to also introduce a new functionality. This can happen either because a new goal does not naturally belong within any of the existing groupings or if a grouping becomes too large, so there is a need to divide it. For example, in the process of developing the above scenario, the new goal *Inform Customer* was identified. This was considered for inclusion with the goals in the *Online Interaction* functionality. However, since informing the customer could be

done either via the web or by email, and since email is not an *Online Interaction*, it was decided to have two different functionalities for interacting with the customer, and a *Customer Contact* functionality was introduced.

4.3.1 GOAL STEP DETAILS

We also attach to each GOAL step, the name of the functionality it belongs to, the information that is used, and the information that is produced or written. So for the step *Arrange delivery*, we also attach the name of the functionality (Delivery Handling), the information used (Order record) and the information produced (none). For example²,

Num.	Step type	Step	Functionality	Data used and produced
5.	GOAL	<i>Arrange delivery</i>	Delivery Handling	Customer Order Customer Order

Below we show the steps of the **Query Late Books scenario**, with the full information for each step.

Electronic Bookstore: Case study				
Fully developed steps for the <i>Query Late Books</i> scenario:				
Key for functionality and data abbreviations:				
DH Delivery Handling				
LGM Lost Goods Management				
CC Customer Contact				
SM Stock Management				
Cust. Info.: Customer Information				
Cust. Order: Customer Order				
Del. Problem: Delivery Problem				
	Step type	Step	Functionality	Data used and produced
1	GOAL:	<i>Determine delivery status</i>	DH	Cust. Order none
2	GOAL:	<i>Log delivery problems</i>	LGM	Cust. Order Del. Problem
3	ACTION:	<i>Request delivery tracking</i>	LGM	Cust. Order Del. Problem

²In order to avoid tables that extend past the width of the page, we put the data used on the first line and the data produced on a second line.

4	GOAL:	<i>Inform customer</i>	CC	Del. Problem Cust. Info.
5	OTHER:	<i>Await response</i>		
6	PERCEPT:	<i>tracking info received</i>		
7	GOAL:	<i>Arrange delivery</i>	DH	Cust. Order Cust. Order
8	GOAL:	<i>Log books outgoing</i>	SM	Book Info Stock DB
9	GOAL:	<i>Inform customer</i>	CC	Cust. Order Cust. Info.
10	GOAL:	<i>Update delivery problem</i>	LGM	Track. Resp. Cust. Order (used) Del. Problem

As can be seen, the above scenario is in two pieces. After an action requesting tracking of the delivery, nothing more can be done until a response is received (or a sufficient delay has occurred to conclude that no response will be received). The scenario could be split into two pieces, with the second piece being triggered by the arrival of the tracking information, or a sufficient time delay. We have chosen to keep it as a single scenario in order to more easily see the entire chain of events arising from the initial trigger. We have used the step type 'OTHER' in order to allow us to indicate the delay and to clarify that there is a wait at this point in the scenario.

4.3.2 CAPTURING ALTERNATIVE SCENARIOS

Scenarios describe a single sequence of steps that can occur. We partially capture alternatives using two techniques. The first technique is that with each scenario we include a description of possible alternatives. These are usually minor variations on the scenario that can be easily described.

Electronic Bookstore: Case study

Alternatives for the *Query Late Books* scenario:

Alternative 1: Book is within or close to expected estimate for delivery time so no tracking is requested and steps 6 to 9 are deleted.

Alternative 2: The tracking information received indicates that the book is on its way, so there is no need for a new delivery. Steps 7 and 8 are deleted.

Alternative 3: After a delay (and possible further requests and delays), no response is received to the tracking request. This situation replaces the percept at step 6, but other steps remain the same.

The second technique for capturing alternatives is to use a collection of scenarios that all relate to a single underlying process. For example, one scenario might describe a normal book order (with minor variations) and another might describe a book order in which the book cannot be obtained and the order is refunded.

Scenarios are intended to give a clearer idea of the system, not to fully define it. Given that there is no attempt to describe all possible scenarios, it may be difficult to know when enough have been developed. There should be a scenario written for at least one version of each important process within the system. A good check is to ensure that all goals, actions and percepts are included in at least one scenario. It is often useful to develop one or two scenarios showing the system functioning when something unexpected happens or an error occurs, to give a sense of how these things will be handled. However, it is usually not productive to define large numbers of exceptional scenarios, as these can clutter the design document and actually make it more difficult to understand the planned system.

Electronic Bookstore: Case study

The list of scenarios developed for the electronic bookstores is as follows:

- Book finding scenario
- Order book scenario
- Pending order arrives scenario
- Stock order scenario
- Stock arrival scenario
- Stock delayed scenario
- Missed stock arrival scenario
- Query late books scenario
- Order status query scenario
- Customer profile update scenario
- WWW site arrival scenario
- Cheaper price notification scenario
- New catalogue scenario

4.4 INTERFACE DESCRIPTION

Agent systems are typically situated in a changing and dynamic environment that can be affected, though not totally controlled, by the agent system. An early question that must be answered is, how is the agent system going to interact with the environment? Specifically,

what input about the environment will be available to the agent system while it is running; and what will the agent system do to interact with and affect the environment? In line with standard texts on agents (Russell and Norvig 1995), we call the incoming information ‘percepts’ and the mechanisms for affecting the environment ‘actions’.

4.4.1 PERCEPTS AND ACTIONS

Percepts often require some processing in order to extract the information that is of value to the agent system. For example, vision frames from a robot camera are not in themselves what the agent system needs to reason about. They must first be processed both to extract the symbolic data, such as ‘ball at 15, 10; robot at 25, 70’ and to extract significant information from this symbolic data. If there is a stream of incoming data, the agent system may want to react only when there is an event of some significance, often determined by change from the previous situation. For example, if a soccer-playing robot sees the ball when it did not do so previously, this is significant. Changes from expectations can also indicate significance. For example, if our robot turns towards where it expects to see the ball and does not see it, this is also significant.

In planning and designing the agent system around the percepts, the designer must take into consideration how data is obtained – does it just arrive, or must the system actively seek the data – as well as the exact nature of the data, and to what extent it can be processed to provide information of interest. It is extremely important to investigate and experiment early on with the exact nature of the percepts available to the system. It is not uncommon for a whole system to be designed and implemented, only to find that the quality of the incoming percepts simply does not allow for reliable provision of the information required by the system.

In systems where percepts originate from physical sensing devices of some sort, the data often contains significant amounts of noise, which may require the use of techniques to filter and cleanse the data. This may be done outside the agent system, however, these issues must be considered carefully at design stage and the developer must establish exactly what it is possible to provide as percepts to the system.

Actions may also be complex, requiring significant design and development outside the realm of the reasoning system, possibly including monitoring for failure or continual feedback loops. This is especially true when manipulation of physical effectors is involved. For example, a system may be designed to have an action *move*, with parameters giving distance and speed. This action will require complex feedback loops within the effector system to enable this action to be carried out. Even electronic actions such as sending a message typically rely on additional low-level code, although this is usually provided in communication interfaces and is not something the developer needs to consider.

Typically, an application has some number of obvious percepts and actions that are the initial definition of how the system will interact with the environment. As the specification is developed, the required interface also becomes further defined. It is extremely important to ensure that these interfaces are correct and/or achievable.

In the electronic bookstore, some initial percepts and actions identified were

- WWW page display (action)
- Bank transaction (action)

- Deliver book (action)
- Arrival at WWW site (percept)
- Bank transaction response (percept)
- Stock arrival (percept)
- User input (percept).

As the specification developed, particularly with the development of scenarios, additional necessary actions and percepts were noted, and some were modified. For example, the electronic system was not able to actually *Deliver book*, so this was changed to an action *Place delivery request*, which sent the delivery request to a courier or to the postal room. The full list of percepts and actions is given below.

Electronic Bookstore: Case study

Following are the percepts and actions identified for the electronic bookstore.

Percepts

- Arrival at WWW site
- Bank transaction response
- Cheaper price report
- Failed stock arrival
- Tracking info
- No tracking response
- Stock arrival
- Stock order delay
- New catalogue
- User input

Actions

- Bank transaction
- E-mail stock order
- Place delivery request
- Request delivery tracking

- Send e-mail
- WWW page display.

As the design develops, decisions may be made to pre-process percepts to provide a number of different percepts to the system. For example, a fairly generic percept such as ‘user input’ might be replaced by a collection of more specific percepts such as ‘user book query’, ‘user book order’, and so on. The original incoming percepts should still be documented, and information should be provided as to what processing is required. However, they may no longer be a direct part of the interface to the agent system.

At this stage, we simply develop lists of percepts and actions, but leave the descriptors and the details until Architectural Design. However, notes may be kept and prototyping done, to determine feasibility.

4.4.2 DATA

As scenarios and functionalities are developed, it is also important to note the data that is a part of these. While developing functionalities and scenarios we note both data that is produced and data that is used. An example of data produced by the Stock Management functionality is the *Stock database*, while an example of data used in a scenario is the *Customer order* data used in the first step of the *Query Late Books* scenario by the GOAL step *Determine delivery status*.

At the system specification phase, we need to especially note any data that is external to the agent system, as this forms part of the interface and should be specified at this stage.

An additional part of the system interface that may need to be specified is the interaction with any other software.

Electronic Bookstore: Case study

In the electronic bookstore, we identify two external data stores that we will use: *Courier DB* and *Postal DB*. These contain information about courier companies’ areas/rates, and postal rates respectively.

In addition, we identify the following clusters of information, without as yet defining all their details:

- Customer DB – contains information about customers, their profile, their history of visits to the site and orders.
- Customer Orders – contains records of orders that have been (fairly recently) sent.

- Pending Orders – contains records of orders that have been placed but not yet sent.
- Delivery Problems – contains records of queries about items that have not arrived and the investigation of these situations.
- Books DB – contains a comprehensive listing of books, with information on suppliers, prices, and so on. Not all books are necessarily stocked.
- Stock DB – contains records of books that are stocked.
- Stock Orders – contains records of stock orders placed and awaiting delivery.

4.5 CHECKING FOR COMPLETENESS AND CONSISTENCY

One of the advantages of doing design in a structured way is that it becomes possible to specify a range of checks for consistency and completeness. Many of these can actually be automated, as is done for a number of them in the prototype Prometheus Design Tool.

NAMES SHOULD BE CONSISTENT

One of the most basic but nevertheless important checks is that names should be consistent. A goal that is named *credit check* in one place in the design, should not be called *check credit* in another place. While the designer may think the equivalence is obvious, it can be very confusing for the person trying to understand the design – and certainly it does not facilitate straightforward mapping to implementation. Having a design tool greatly simplifies maintaining naming consistency.

EVERY GOAL IN A SCENARIO AND FUNCTIONALITY

At the system specification phase, the main checks that can be done apart from naming consistency are to ensure that the system is adequately specified with respect to the system goals. It is desirable for all goals to be in some way covered in a scenario. There are three ways a goal can be covered:

- It can be specifically mentioned as a step or a trigger.
- All its sub-goals may be mentioned specifically.
- Its parent goal may be mentioned specifically.

All goals that are not labelled as abstract goals should also be contained in some functionality. This is needed to ensure that they are addressed as the design progresses.

SCENARIOS AND FUNCTIONALITIES FOR EVERY ACTION AND PERCEPT

All percepts and actions should also be shown somewhere in a scenario, and should belong to at least one functionality. If they are missing from the scenarios, then it is possible that some significant behaviour has been overlooked. However, it may also be that it is mentioned in a scenario variation, rather than being specifically covered. For example, if the *bank transaction response* percept had been modelled as an *accept response* and a *reject response*, it is quite possible that the *reject response* would occur only in a scenario variation, rather than in a scenario step.

Checking at this phase is really a matter of drawing the attention of the developer to things that may have been overlooked, rather than identifying things that are clearly wrong or inconsistent. Nevertheless, automated support for this can be very helpful.